4脚口ボットでボウリング

OmniCampus ID：60587 2026年10月28日

1. 取り組んだ課題

Go2が球の前まで歩き、右前足で蹴って6本のピンを倒すタスクを作成した。歩行を使った移動に、足で物を動かす動作を加えたいと考え、ボウリングを選んだ。ピンを倒すだけでなく、蹴った後も口ボットが立っていることを目標とした。

2. 制御と学習

シミュレータにはGenesisを使った。球の位置はシミュレータから取得し、球の手前で止まるように速度指令を与える。歩行にはPPOで学習した方策を使用した。入力は胴体の姿勢・角速度、関節の角度・速度、速度指令など45値で、12関節の目標角度の補正量を出力する。

キックでは足先の軌道を手設計し、逆運動学（IK）で関節角度に変換してPD制御で追従させた。球は足との接触で転がり、ピンとの衝突はGenesisが計算する。歩行とキックに加え、後述する姿勢の崩れに対応するため、蹴った後の立位維持にも別のPPO方策を用意した。

学習対象	条件
歩行・停止	基本歩行100反復、横方向の外乱を加えて150反復、停止・速度変更・旋回を含む指令で500反復。計750反復。1,024並列環境、24ステップ / 反復、seed=1。
キック後の立位維持	歩行方策を初期値として追加学習。キック後に記録した79姿勢と通常の立位を初期状態とし、速度は0にして開始した。512並列環境、24ステップ / 反復、seed=2。450反復まで学習し、投球で評価した350反復時点を採用した。
共通の設定	Actor・Criticは512→256→128のMLP（ELU）。PPOは5 epoch、4 minibatch。制御は50 Hz、物理計算は2サブステップ。報酬やその他の設定値は同梱の設定ファイルに記載した。

3. うまくいかなかった点と修正

最初は前足を上げて振るだけだったため、支持を失って転倒した。そこで、足を上げる前に胴体の目標位置を後方6 cm、左12 cm、下1.5 cmへ移し、残りの3脚で支えるようにした。前足の目標軌道は約33.5 cm、振り出し時間は0.16秒とした。

この変更で6本を倒せたが、蹴った後に胴体を戻す区間で約20°傾いた。復帰用の方策には、胴体の傾き・角速度・高さのずれ・急な関節動作を抑える報酬を与えた。150反復の後に傾きへの罰則を強め、さらに200反復したモデルを採用した。切り替えも見直し、キック開始1.5秒後から0.3秒かけて復帰用の方策へ移るようにした。

4. 結果

球は質量1.0 kg、半径0.12 m、ピンは各0.10 kg、高さ約0.315 mとした。6本のピンを三角形に並べ、側面にバンパーのある5.6×1.4 mのレーンを使った。鉛直から45°を超えて傾いたピンを倒れたものとして数えた。

比較した段階	観察した動作	倒した本数
歩行の学習前	球の前まで到達できなかった。	0 / 6
歩行のみ	球の前まで歩いて停止した。キックは行わない。	0 / 6
キックの試作	前足を振ろうとして姿勢を崩し、転倒した。	0 / 6
完成版	6本を倒し、蹴った後も立位を保った。	6 / 6

球とピンの配置、開始姿勢、全景カメラをそろえて比較した。歩行のみ・キックの試作・完成版では同じ歩行方策を使い、キックの制御を変更している。完成版には復帰用の方策も加えているため、この4段階は学習回数だけの比較ではない。

完成版では、終了時にも6本すべてが倒れていた。ロボットの最大傾きは20.4°から7.4°に減り、終了時は0.28°だった。ここでの傾きはロール・ピッチの絶対値の大きい方である。追加学習と制御の切り替えを同時に変えたため、この数値だけで学習単独の効果は判断できない。

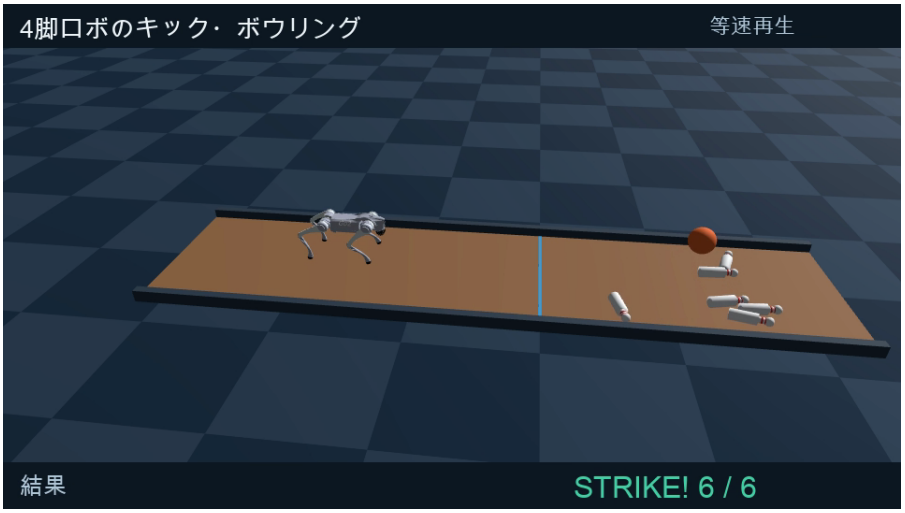


図1 完成版の投球後。6本のピンが倒れ、Go2は立位を保っている。

5. 今回確認できていないこと

本編では球の位置とレーンを固定し、球の座標も既知としている。カメラを使って位置を求める実験は第8節に分けた。各条件で何度も投球したわけではないため、安定して成功する確率までは分からない。次は同じ位置で試行を繰り返し、その後に照明や摩擦も変えて確かめたい。実機ではまだ試していない。

6. 評価項目との対応

創造性

公式例で学習する歩行を使って、前足で球を蹴るボウリングを作った。追加したのは、球の手前で止まる処理、足を振る軌道、蹴った後の姿勢を戻す処理である。ピンの本数だけを見ると転倒した投球も成功に見えてしまうため、最後まで立っているかも確認した。

技術力

歩行と復帰はPPO、足先の軌道はIK、関節の駆動はPD制御で動かした。前足を上げるだけでは転倒したので、先に体重を移す処理を入れた。6本を倒せるようになってからも、蹴った後に胴体が沈み込んだため、復帰用の追加学習と切り替え時刻の調整を行った。接触する瞬間よりも、その前後の姿勢をどう保つかが問題になった。

実用性

応用先として考えているのは、床上の軽い物を足で押して位置を直すような作業である。その場合も、物の手前で止まり、片足を上げて接触し、最後に姿勢を戻す必要がある。今回は球を対象に、この流れをシミュレーションで試した。

カメラで位置を求める別版では、球をずらしても蹴れたが、全ピンが倒れるとは限らなかった。対象の形や重さを変えた場合はまだ調べていない。実際の作業に使うには、実力メラでの検出、接触力の管理、実機での確認が必要になる。現状では、歩行から物体への接触に移るときの失敗と修正を比べる教材として使うことを考えている。

表現力

成功した投球だけでは変更点が分かりにくいため、動画には歩行の学習前、歩行のみ、転倒した試作、完成版を入れた。比較する場面は同じ全景で撮影した。足の動きは横から1/4倍速で見せ、最後に全景へ戻して本数と姿勢を確認する構成にした。

7. 実行環境・コード・参考資料

Apple M5 Pro（16コアGPU、24 GBメモリ）でMetal / MPSを使用した。Python 3.12、Genesis 1.4.5、PyTorch 2.9.1、rsl-rl 5.5.1で動作を確認した。

実行手順はREADME.mdに記載した。kick_bowling.pyがタスク実行、train_recovery.pyが復帰用方策の学習を担当する。歩行方策はmodels/locomotion.pt、復帰用方策はmodels/recovery.ptに保存した。提出動画は1分32.4秒で、全景と横視点は同じエピソードから録画している。

歩行環境はGenesisの公式例を基にした。原本とApache-2.0ライセンスはvendor/、変更点はTHIRD_PARTY.mdに記載した。

[1] Genesis World, Go2 locomotion example
[2] rsl-rl（PPO実装、BSD-3-Clause）

8. 追加実験：カメラで球の位置を求める

既知の球座標を使う本編とは別に、固定RGB-Dカメラから位置を求める版を作成した。球を置く位置を変えても、その場所まで近づいて蹴れるかを調べた。

640×480のRGB画像でオレンジ色の領域を抽出し、輪郭に楕円を当てはめて中心を求める。カメラの校正情報と、床面・球半径から分かる球の中心高さを使って、画像座標をレーン上の位置へ変換する。深度画像にも球面を当てはめ、両者が合わない場合は投球を開始しない。球の真の座標は誤差の評価にだけ使い、接近やキックの制御には渡していない。

推定した球位置に合わせて停止位置と足先軌道を移動させた。歩行と復帰の方策は本編と同じで、画像処理には色と幾何を使っている。球が静止している状態で最初に1回検出する方式であり、移動中の球を追跡するものではない。

前後・左右の位置をそれぞれ−4、0、+4 cmずらした9条件を、各1回撮影した。下表は倒れた本数である。

前後のずれ / 左右のずれ	−4 cm	0 cm	+4 cm
−4 cm	3 / 6	5 / 6	4 / 6
0 cm	4 / 6	6 / 6	6 / 6
+4 cm	4 / 6	4 / 6	3 / 6

9条件すべてで検出・接近・キック・立位維持を確認した。6本すべて倒せたのは2条件だった。さらに、調整に使っていない3位置でも試した。

前後 / 左右のずれ	推定位置の誤差	倒した本数	終了時
+1.73 / -2.34 cm	0.67 mm	3 / 6	立位維持
-3.06 / +1.22 cm	0.45 mm	4 / 6	立位維持
+2.64 / +3.27 cm	0.50 mm	3 / 6	立位維持

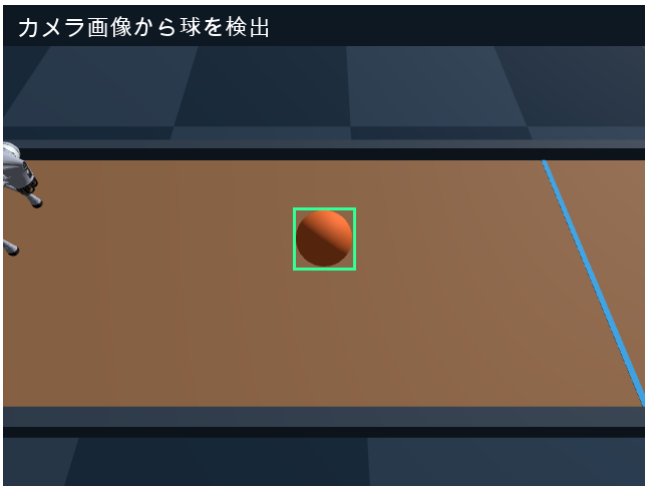


図2 カメラ画像から検出した球。緑枠が画像処理の検出領域。

位置を変えても球には当たったが、6本すべて倒せたのは12試行中2回だった。球の位置が分かることと、狙いどおりに蹴れることは分けて改善する必要がある。今回のカメラ画像にはノイズを加えていない。球の色・半径、床の高さ、照明、カメラ位置も固定しており、実カメラで同じ精度が出るかは未確認である。動画と全12試行の記録はGitHubに保存した。